

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина: Методы трансляции

ОТЧЕТ
к лабораторной работе №1
на тему

**ОПРЕДЕЛЕНИЕ МОДЕЛИ ЯЗЫКА. ВЫБОР ИНСТРУМЕНТАЛЬНОЙ
ЯЗЫКОВОЙ СРЕДЫ**

Студент
Преподаватель

Е. С. Кахновский
Н. Ю. Гриценко

Минск 2024

СОДЕРЖАНИЕ

1 Цель работы	3
2 Определение подмножества языка программирования C#	4
3 Определение инструментальной языковой среды	10
Заключение	11
Список использованных источников	12
Приложение А (обязательное) Листинг кода	13

1 ЦЕЛЬ РАБОТЫ

Определить подмножество языка программирования (типы констант, переменных, операторов и функций). Определить инструментальную языковую среду, которая включает в себя язык программирования и операционную систему. Привести подробное описание инструментальной языковой среды, а также тексты 2-3 программ, включающих элементы этого подмножества.

2 ОПРЕДЕЛЕНИЕ ПОДМНОЖЕСТВА ЯЗЫКА ПРОГРАММИРОВАНИЯ C#

Литерал в языке программирования C# представляет собой фиксированное значение, записанное непосредственно в исходном коде программы. Литералы используются для представления константных значений различных типов данных, таких как числа, символы, строки и булевы значения (рисунок 2.1). Они представляют собой непосредственные данные, которые компилятор может интерпретировать и обрабатывать.

Более подробное описание различных типов литералов в C#:

1 Целочисленные литералы представляют собой последовательность цифр без десятичной точки. Они могут быть записаны в различных системах счисления, таких как десятичная (12345), двоичная (0b0110), восьмеричная (0752) или шестнадцатеричная (0xAF).

2 Литералы с плавающей точкой представляют собой числа с десятичной точкой или экспонентой. Они могут быть записаны в форме десятичной (3.14) или экспоненциальной нотации (3.0e7).

3 Символьные литералы представляют собой одиночные символы, заключенные в одинарные кавычки ('W').

4 Строковые литералы представляют собой последовательность символов, заключенных в двойные кавычки ("Hello, World!").

5 Булевы литералы представляют собой логические значения *true* или *false*.

6 Литерал *null* используется для представления значения *null*.

```
// Целочисленные константы
int intValue = 42;
long longValue = 1234567890L;
short shortValue = 32767;
byte byteValue = 255;

// Числа с плавающей точкой
float floatValue = 3.14f;
double doubleValue = 123.456;

// Текстовая константа
string stringValue = "Hello, C#!";
```

Рисунок 2.1 – Литералы языка C#

Целочисленные типы представляют целые числа. Все целочисленные типы являются типами значений. Они также простые типы и могут быть инициализированы с помощью литерала (рисунок 2.2). Все целочисленные типы поддерживают арифметические операторы, побитовые логические операторы, операторы сравнения и равенства.

```
C#  
  
var decimalLiteral = 42;  
var hexLiteral = 0x2A;  
var binaryLiteral = 0b_0010_1010;
```

Рисунок 2.2 – Целочисленные литералы

Числовые типы с плавающей запятой представляют действительные числа. Все числовые типы с плавающей запятой являются типами значений. Они также представляют собой простые типы и могут быть инициализированы литералами (рисунок 2.3). Все числовые типы с плавающей запятой поддерживают арифметические операторы, а также операторы сравнения и равенства.

C# поддерживает следующие предварительно определенные типы с плавающей запятой:

- 1 *Float*. Точность составляет 6-9 цифр, размер занимает 4 байта.
- 2 *Double*. Точность составляет 15-17 цифр, размер занимает 8 байт.
- 3 *Decimal*. Точность составляет 28-29 цифр, размер занимает 16 байт.

```
C#  
  
double d = 3D;  
d = 4d;  
d = 3.934_001;  
  
float f = 3_000.5F;  
f = 5.4f;  
  
decimal myMoney = 3_000.5m;  
myMoney = 400.75M;
```

Рисунок 2.3 – Литералы с плавающей точкой

Строка – это объект типа *string*, значением которого является текст (рисунок 2.4). Внутри программы текст хранится в виде упорядоченной коллекции объектов *char* только для чтения. В конце строки С# нет символа, завершающего значение *null*; поэтому строка С# может содержать любое количество внедренных символов *null*.

```
C#

// Declare without initializing.
string message1;

// Initialize to null.
string message2 = null;

// Initialize as an empty string.
// Use the Empty constant instead of the literal "".
string message3 = System.String.Empty;

// Initialize with a regular string literal.
string oldPath = "c:\\Program Files\\Microsoft Visual Studio 8.0";
```

Рисунок 2.4 – Объекты типа *string*

Для выполнения логических операций со значениями типа *bool* используйте логические операторы. Тип *bool* является типом результата операторов сравнения и равенства (рисунок 2.5). Выражение *bool* может быть управляющим условным выражением в операторах *if*, *do*, *while* и *for* и условном операторе “?:”. Значение по умолчанию для типа *bool* – *false*. Можно использовать литералы *true* и *false* для инициализации переменной *bool*.

```
C#

bool check = true;
Console.WriteLine(check ? "Checked" : "Not checked");

Console.WriteLine(false ? "Checked" : "Not checked");
```

Рисунок 2.5 – Тип *bool*

В унифицированной системе типов С# все типы, стандартные и определяемые пользователем, ссылочные типы и типы значений напрямую или косвенно наследуются из *object*.

Переменной типа *object* можно назначать значения любого типа. Любой переменной *object* можно назначить значение по умолчанию с помощью литерала *null*.

Когда переменная типа значения преобразуется в объект, она будет указана в поле. Если переменная типа *object* преобразуется в тип значения, он считается распакованным [1].

В языке C# существует различные структуры данных, подходящие под различные задачи:

1 Связный список (*Linked List*) представляет набор связанных узлов, каждый из которых хранит собственно данные и ссылку на следующий узел. Связные списки могут различаться. Есть односвязные списки, в которых каждый узел хранит указатель только на следующий узел. Есть двусвязные списки: в них каждый элемент хранит ссылку как на следующий элемент, так и на предыдущий. Есть кольцевые замкнутые списки.

2 Стек (*stack*) является структурой данных, которая работает по принципу "последним пришел, первым вышел" (*Last In, First Out, LIFO*). Это означает, что элементы, добавленные в стек последними, будут удалены из него первыми.

3 Очередь (*queue*) – это структура данных, которая работает по принципу *FIFO* (*First In First Out* – Первый пришел, первый вышел). При добавлении новый элемент помещается в конец очереди или ее хвост, а удаление идет с начала очереди или головы очереди.

4 Массив (*array*) представляет набор однотипных данных. Объявление массива похоже на объявление переменной за тем исключением, что после указания типа ставятся квадратные скобки.

5 Словарь (*dictionary*) хранит объекты, которые представляют пару ключ-значение. Класс словаря *Dictionary* типизируется двумя типами: параметр “K” представляет тип ключей, а параметр “V” предоставляет тип значений.

Цикл состоит из условия и блока кода, который выполняется, пока условие истинно. Каждое выполнение цикла называется итерацией. Операторы циклов в языке C# включают в себя операторы *while*, *do-while* и *for*.

Операторы *break* и *continue* используются для управления выполнением цикла. Цикл *for* применяется для повторного выполнения блока кода определенное количество раз. Цикл *while* выполняет блок кода до тех пор, пока условие истинно, проверяя его перед каждой итерацией. Цикл *do-while* похож на цикл *while*, но условие проверяется после каждой итерации, гарантируя выполнение тела цикла хотя бы один раз.

Методы содержат собой набор инструкций, которые выполняют определенные действия. По сути метод – это именованный блок кода, который выполняет некоторые действия. Перед названием метода идет возвращаемый тип данных. После названия метода в скобках идет перечисление параметров. После списка параметров в круглых скобках идет блок кода, который представляет набор выполняемых методом инструкций.

Рекурсивная функция представляет такую конструкцию, при которой функция вызывает саму себя.

Локальные функции представляют функции, определенные внутри других методов. Локальная функция, как правило, содержит действия, которые применяются только в рамках ее метода [2].

Класс представляет собой шаблон, по которому определяется форма объекта. В нем указываются данные и код, который будет оперировать этими данными. В С# используется спецификация класса для построения объектов, которые являются экземплярами класса. Следовательно, класс, по существу, представляет собой ряд схематических описаний способа построения объекта.

При этом очень важно подчеркнуть, что класс является логической абстракцией. Физическое представление класса появится в оперативной памяти лишь после того, как будет создан объект этого класса.

Классы и структуры – это, по сути, шаблоны, по которым можно создавать объекты. Каждый объект содержит данные и методы, манипулирующие этими данными. При определении класса объявляются данные, которые он содержит, а также код, оперирующий этими данными.

Если самые простые классы могут содержать только код или только данные, то большинство настоящих классов содержит и то и другое [3].

В языке программирования С# операторы разделены на арифметические, логические и сравнительные операции.

1 Арифметические операторы включают сложение (+), вычитание (-), деление (/), умножение (*), деление по модулю (%), инкремент (++), декремент (--). Инкремент и декремент могут быть как постфиксными, так и префиксными.

2 Логические операторы включают логическое И (&&), логическое ИЛИ (||), логическое НЕ (!).

3 Операторы сравнения включают оператор равенства (==), оператор неравенства (!=), оператор больше (>), оператор меньше (<), оператор больше или равно (>=), оператор меньше или равно (<=). Также в С# присутствует тернарный оператор, который позволяет выбирать одно из двух возможных действий на основе значения логического выражения.

4 Операции присваивания включают присваивание (`=`), присваивание после сложения (`+=`), присваивание после вычитания (`-=`), присваивание после умножения (`*=`), присваивание после деления (`/=`), присваивание после получения остатка от деления (`%=`), присваивание после поразрядной конъюнкции (`&=`), присваивание после поразрядной дизъюнкции (`|=`), присваивание после операции исключающего ИЛИ (`^=`).

5 Операторы доступа к элементам включают оператор квадратных скобок (`[]`), оператор точки (`.`), оператор двойной точки (`::`).

6 Операторы побитовых операций включают оператор инверсии битов (`~`), оператора логического сдвига влево (`<<`), оператора логического сдвига вправо (`>>`).

Ключевое слово *using* используется для интеграции библиотек. Оно позволяет включить содержимое сторонних библиотек, делая их заголовочные файлы доступными для использования в программе. Это обеспечивает возможность обращения к функциям, классам и другим компонентам библиотеки.

Условный оператор в языке C# используется для выбора выполнения определенного действия в зависимости от условия. Если условие, указанное после ключевого слова *if*, истинно, выполняется один блок кода, в противном случае – другой блок кода, указанный после ключевого слова *else*.

Подмножество условных операторов в языке C# включает в себя операторы *if*, *else if*, *else*, а также операторы *switch case*.

3 ОПРЕДЕЛЕНИЕ ИНСТРУМЕНТАЛЬНОЙ ЯЗЫКОВОЙ СРЕДЫ

Python 3.12 – это язык программирования, который широко используется в интернет-приложениях, разработке программного обеспечения, науке о данных и машинном обучении (ML). Разработчики используют *Python*, потому что он эффективен, прост в изучении и работает на разных платформах.

Программы на языке *Python* можно скачать бесплатно, они совместимы со всеми типами систем и повышают скорость разработки. *Python* является интерпретируемым языком, то есть он выполняет код построчно. Если в коде программы присутствуют ошибки, она перестает работать. Это позволяет программистам быстро найти ошибки в коде.

Программистам не нужно объявлять типы переменных при написании кода, потому что *Python* определяет их во время выполнения. Эта функция позволяет писать программы на *Python* значительно быстрее.

Python ближе к естественным языкам, чем ряд других языков программирования. Благодаря этому программистам не нужно беспокоиться о его базовой функциональности, например об архитектуре и управлении памятью.

Python рассматривает все элементы как объекты, но также поддерживает другие типы программирования (например, структурное и функциональное программирование).

Библиотека – это набор часто используемых кодов, которые разработчики могут включать в свои программы *Python*, чтобы не писать код с нуля. По умолчанию в *Python* доступна стандартная библиотека, которая содержит большое количество многократно используемых функций. Кроме того, доступно более 137 000 библиотек *Python* для различных задач, в числе которых интернет-разработка, наука о данных и машинное обучение.

Платформы *Python* – это наборы пакетов и модулей. Модуль – это набор связанного кода, а пакет – это набор модулей. Разработчики могут использовать платформы *Python* для более быстрого создания приложений *Python*, поскольку им не нужно беспокоиться о низкоуровневых деталях (например, скорости обмена данных в веб-приложении) или том, как *Python* ускоряет работу программы [4].

Выбрана *Visual Studio Code* в качестве редактора кода. Операционная система, используемая в данной среде, – Windows. Компьютер – PC.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы было определено подмножество языка C#. Определена инструментальная языковая среда, которая включает в себя язык программирования и операционную систему.

Приведено подробное описание инструментальной языковой среды, а также тексты 2 программ, включающих элементы этого подмножества.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Типы значений [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/csharp/language-reference/builtin-types/value-types>.

[2] Структуры данных [Электронный ресурс]. – Режим доступа: <https://metanit.com/sharp/algorithm/2.1.php>.

[3] Классы [Электронный ресурс]. – Режим доступа: https://professorweb.ru/my/csharp/charp_theory/level5/5_1.php.

[4] Что такое *Python* [Электронный ресурс]. – Режим доступа: <https://aws.amazon.com/ru/what-is/Python>.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг кода

Листинг 1 – файл *Example1.cs*.

```
using System;

class Program
{
    static void Main(string[] args)
    {
        int sumInt = 5 + 3;
        float sumFloat = 3.5f + 2.5f;

        int differenceInt = 10 - 4;
        float differenceFloat = 7.5f - 2.3f;

        int divisionInt = 20 / 4;
        float divisionFloat = 15.0f / 3.0f;

        int multiplicationInt = 6 * 4;
        float multiplicationFloat = 3.0f * 2.5f;

        int modulusInt = 10 % 3;

        int num1 = 5;
        num1++;
        num1--;

        bool isTrue = true;
        bool isFalse = false;

        string message = "Hello, ";
        string name = "John";
        string greeting = message + name;

        if (sumInt > 0)
        {
            Console.WriteLine("sumInt больше нуля.");
        }

        int switchValue = 2;
        switch (switchValue)
        {
            case 1:
                Console.WriteLine("switchValue равно 1.");
                break;
            case 2:
                Console.WriteLine("switchValue равно 2.");
                break;
            default:
                Console.WriteLine("switchValue не равно ни 1, ни 2.");
                break;
        }

        object obj1 = sumInt;
    }
}
```

Листинг 2 – файл *Example2.cs*.

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main(string[] args)
    {
        List<int> list = new List<int>() { 1, 2, 3, 4, 5 };

        Stack<int> stack = new Stack<int>();
        stack.Push(1);
        stack.Push(2);
        stack.Push(3);

        Queue<string> queue = new Queue<string>();
        queue.Enqueue("first");
        queue.Enqueue("second");
        queue.Enqueue("third");

        int[] array = new int[] { 10, 20, 30, 40, 50 };

        Dictionary<string, int> dictionary = new Dictionary<string, int>();
        dictionary.Add("one", 1);
        dictionary.Add("two", 2);
        dictionary.Add("three", 3);

        for (int i = 0; i < 5; i++)
        {
            Console.WriteLine("for: " + i);
        }

        int j = 0;
        while (j < 5)
        {
            j++;
        }

        int k = 0;
        do
        {
            k++;
        } while (k < 5);

        RecursiveFunction(5);

        void LocalFunction()
        {
            Console.WriteLine("This is a local function.");
        }
        LocalFunction();

        bool logicalAnd = true && false;
        bool logicalOr = true || false;
    }
}
```

```

bool logicalNot = !true;

bool equal = (10 == 10);
bool notEqual = (20 != 30);
bool greaterThan = (40 > 30);
bool lessThan = (50 < 60);
bool greaterThanOrEqual = (70 >= 70);
bool lessThanOrEqual = (80 <= 90);

int x = 5;
int y = (x > 0) ? 1 : -1;

int num = 10;
num += 5; // Прибавление
num -= 3; // Вычитание
num *= 2; // Умножение
num /= 4; // Деление
num %= 3; // Остаток от деления

int[] numbers = { 1, 2, 3, 4, 5 };
Console.WriteLine("Элемент по индексу 2: " + numbers[2]);

string str = "Hello";
Console.WriteLine(str.Length);

}

static void RecursiveFunction(int n)
{
    if (n > 0)
    {
        RecursiveFunction(n - 1);
        Console.WriteLine("Recursive: " + n);
    }
}
}

```